TRANSPORT AND TELECOMMUNICATION INSTITUTE

ENGINEERING FACULTY

Laboratory work N4

Course: **Programming Languages Concepts**

Theme: **Strings & Arrays in Programming Language**

Student: Igors Oļeiņikovs

Student code: 93642

Group: 4501BTA

Riga
2026

CONTENTS

1. LABORATORY WORK TASK 3
2. PROGRAM CODE AND RESULTS 3
3. CONCLUSIONS 12
4. REFERENCES 13

1. LABORATORY WORK TASK

The purpose of this laboratory work is to practise string and array operations in a programming language and compare the selected language with other programming languages. Ruby was selected because its syntax is concise and expressive, while still providing clear built-in support for strings, arrays, formatting, validation, and random number generation.

Implemented tasks:

1. Calculate string length.

2. Read a first name and a second name and join them into a single string.

3. Convert number to string and back.

4. Validate a date from day, month, and year and format it as text.

5. Read and store n array elements, display them in reverse order, and calculate the average.

6. Read a 4 x 4 matrix row by row and display the sum of all elements.

7. Generate an array of 40 random integers from 0 to 9 and display the values in 5 columns.

2. PROGRAM CODE AND RESULTS

<u>Task 1. Calculate string length</u>

The program reads a string and calculates its length using Ruby's built-in String#length method.

```
#!/usr/bin/env ruby

$stdout.sync = true

print 'Enter a string: '
input = gets
exit if input.nil?

text = input.chomp

puts "String: #{text}"
puts "Length: #{text.length}"
```

```
src : fish — Konsole
┌[X]─[07:10-15/04]─[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└─$ _6r4ndf4th3r$./task_1_string_length.rb
Enter a string: Lorem ipsum dolor sit amet.
String: Lorem ipsum dolor sit amet.
Length: 27
┌[X]─[07:12-15/04]─[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└─$ _6r4ndf4th3r$
```

Task 2. Concatenate first name and second name

The program reads two strings from the user and joins them into one full name with a space between the parts.

```ruby
#!/usr/bin/env ruby

$stdout.sync = true

def read_non_empty(prompt)
  loop do
    print prompt
    input = gets
    exit if input.nil?

    value = input.strip
    return value unless value.empty?

    puts 'Input must not be empty.'
  end
end

first_name = read_non_empty('Enter first name: ')
second_name = read_non_empty('Enter second name: ')
full_name = "#{first_name} #{second_name}"

puts "Concatenated string: #{full_name}"
```

```
src : fish — Konsole
┌[X]─[07:13-15/04]─[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└─$ _6r4ndf4th3r$./task_2_name_concatenation.rb
Enter first name: Bond
Enter second name: James
Concatenated string: Bond James
┌[X]─[07:14-15/04]─[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└─$ _6r4ndf4th3r$
```

Task 3. Convert number to string and back

The program demonstrates both directions of conversion: Integer#to_s converts a number to a string, and Integer() converts a numeric string back to an integer.

```
#!/usr/bin/env ruby

$stdout.sync = true

def read_integer(prompt)
  loop do
    print prompt
    input = gets
    exit if input.nil?

    value = Integer(input.strip, exception: false)
    return value unless value.nil?

    puts 'Please enter a valid whole number.'
  end
end

def read_numeric_string(prompt)
  loop do
    print prompt
    input = gets
    exit if input.nil?

    value = input.strip
    return value unless Integer(value, exception: false).nil?

    puts 'Please enter a string that contains a valid whole number.'
  end
end

number = read_integer('Enter an integer to convert to a string: ')
number_as_string = number.to_s

numeric_string = read_numeric_string('Enter a numeric string to convert to an
integer: ')
string_as_number = Integer(numeric_string, exception: false)

puts "Integer to string: #{number} -> \"#{number_as_string}\""
puts "String to integer: \"#{numeric_string}\" -> #{string_as_number}"
```

```
┌[X]-[07:15-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└─_6r4ndf4th3r$./task_3_string_integer_conversion.rb
Enter an integer to convert to a string: 123456789
Enter a numeric string to convert to an integer: 987654321
Integer to string: 123456789 -> "123456789"
String to integer: "987654321" -> 987654321
┌[X]-[07:16-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└─_6r4ndf4th3r$
```

Task 4. Format a validated date

The program reads day, month, and year values, validates the date including month-specific day limits and leap years, and formats valid input as number/string/number such as 12th November 1992.

```ruby
#!/usr/bin/env ruby

$stdout.sync = true

MONTH_NAMES = %w[
  January February March April May June
  July August September October November December
].freeze

def read_integer(prompt)
  loop do
    print prompt
    input = gets
    exit if input.nil?

    value = Integer(input.strip, exception: false)
    return value unless value.nil?

    puts 'Please enter a valid whole number.'
  end
end

def leap_year?(year)
  (year % 400).zero? || ((year % 4).zero? && !(year % 100).zero?)
end

def days_in_month(month, year)
```

```
  case month
  when 1, 3, 5, 7, 8, 10, 12
    31
  when 4, 6, 9, 11
    30
  when 2
    leap_year?(year) ? 29 : 28
  else
    0
  end
end

def ordinal_suffix(day)
  return 'th' if (11..13).cover?(day % 100)

  case day % 10
  when 1
    'st'
  when 2
    'nd'
  when 3
    'rd'
  else
    'th'
  end
end

day = read_integer('Enter day: ')
month = read_integer('Enter month: ')
year = read_integer('Enter year: ')

if year <= 0
  puts 'Invalid date: year must be greater than 0.'
  exit 1
end

unless (1..12).cover?(month)
  puts 'Invalid date: month must be between 1 and 12.'
  exit 1
end

max_day = days_in_month(month, year)

unless (1..max_day).cover?(day)
  puts "Invalid date: day #{day} is not valid for #{MONTH_NAMES[month - 1]}
#{year}."
  exit 1
```

```
end

puts "Formatted date: #{day}#{ordinal_suffix(day)} #{MONTH_NAMES[month - 1]}
#{year}"
```

```
src : fish — Konsole
┌─[X]─[07:17-15/04]─[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└─╼ _6r4ndf4th3r$./task_4_date_formatter.rb
Enter day: 25
Enter month: 12
Enter year: 1990
Formatted date: 25th December 1990
┌─[X]─[07:17-15/04]─[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└─╼ _6r4ndf4th3r$./task_4_date_formatter.rb
Enter day: 45
Enter month: 22
Enter year: 90
Invalid date: month must be between 1 and 12.
┌─[X]─[07:19-15/04]─[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└─╼ _6r4ndf4th3r$./task_4_date_formatter.rb
Enter day: 45
Enter month: 11
Enter year: 90
Invalid date: day 45 is not valid for November 90.
┌─[X]─[07:19-15/04]─[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└─╼ _6r4ndf4th3r$./task_4_date_formatter.rb
Enter day: 12
Enter month: 3
Enter year: -9
Invalid date: year must be greater than 0.
┌─[X]─[07:20-15/04]─[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└─╼ _6r4ndf4th3r$
```

Task 5. Reverse an array and calculate the average

The program reads n elements into an array, displays them in reverse order, and calculates the arithmetic mean of the entered numbers.

```
#!/usr/bin/env ruby

$stdout.sync = true

def read_positive_integer(prompt)
  loop do
```

```
    print prompt
    input = gets
    exit if input.nil?

    value = Integer(input.strip, exception: false)
    return value if !value.nil? && value.positive?

    puts 'Please enter a positive whole number.'
  end
end

def read_number(prompt)
  loop do
    print prompt
    input = gets
    exit if input.nil?

    value = Float(input.strip, exception: false)
    return value unless value.nil?

    puts 'Please enter a valid number.'
  end
end

def format_number(number)
  return number.to_i.to_s if number == number.to_i

  format('%.2f', number)
end

element_count = read_positive_integer('Enter number of elements: ')
numbers = Array.new(element_count) do |index|
  read_number("Enter element #{index + 1}: ")
end

reversed_numbers = numbers.reverse
average = numbers.sum / numbers.length

puts "Array: #{numbers.map { |number| format_number(number) }.join(', ')}"
puts "Reversed array: #{reversed_numbers.map { |number| format_number(number)
}.join(', ')}"
puts format('Average: %.2f', average)
```

```
src : fish — Konsole
┌[X]-[07:21-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└-_6r4ndf4th3r$./task_5_reverse_array_average.rb
Enter number of elements: 7
Enter element 1: 54
Enter element 2: 3
Enter element 3: 66
Enter element 4: 8
Enter element 5: 9
Enter element 6: 47
Enter element 7: 42
Array: 54, 3, 66, 8, 9, 47, 42
Reversed array: 42, 47, 9, 8, 66, 3, 54
Average: 32.71
┌[X]-[07:21-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└-_6r4ndf4th3r$
```

Task 6. Calculate the sum of a 4 x 4 matrix

The program reads a 4 x 4 integer matrix row by row, displays it in matrix form, and calculates the sum of all matrix elements.

```ruby
#!/usr/bin/env ruby

$stdout.sync = true

MATRIX_SIZE = 4

def read_matrix_row(row_number)
  loop do
    print "Enter row #{row_number} (#{MATRIX_SIZE} whole numbers separated by
spaces): "
    input = gets
    exit if input.nil?

    values = input.split.map { |token| Integer(token, exception: false) }
    return values if values.length == MATRIX_SIZE && values.none?(&:nil?)

    puts "Please enter exactly #{MATRIX_SIZE} whole numbers separated by spaces."
  end
end

matrix = Array.new(MATRIX_SIZE) { |index| read_matrix_row(index + 1) }
total = matrix.flatten.sum

puts 'Matrix:'
```

```
matrix.each do |row|
  puts row.map { |value| format('%4d', value) }.join(' ')
end
puts "Sum of all elements: #{total}"
```

```
src : fish — Konsole
┌[X]-[07:22-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└-_6r4ndf4th3r$./task_6_matrix_sum.rb
Enter row 1 (4 whole numbers separated by spaces): 5 8 4 3
Enter row 2 (4 whole numbers separated by spaces): 2 8 4 1
Enter row 3 (4 whole numbers separated by spaces): 6 5 4 7
Enter row 4 (4 whole numbers separated by spaces): 6 1 2 3
Matrix:
   5    8    4    3
   2    8    4    1
   6    5    4    7
   6    1    2    3
Sum of all elements: 69
┌[X]-[07:23-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└-_6r4ndf4th3r$
```

Task 7. Generate a random array

The program creates an array of 40 pseudo-random integers in the range 0..9 and prints the values in five columns. The array size and range are declared as constants.

```
#!/usr/bin/env ruby

$stdout.sync = true

ARRAY_SIZE = 40
MIN_VALUE = 0
MAX_VALUE = 9
COLUMN_COUNT = 5

seed_text = ENV['LAB4_RANDOM_SEED']
seed = Integer(seed_text, exception: false)
rng = seed.nil? ? Random.new : Random.new(seed)

numbers = Array.new(ARRAY_SIZE) { rng.rand(MIN_VALUE..MAX_VALUE) }

puts "Generated #{ARRAY_SIZE} random integers in range #{MIN_VALUE}..#{MAX_VALUE}:"
numbers.each_slice(COLUMN_COUNT) do |row|
  puts row.map { |value| format('%3d', value) }.join(' ')
```

```
end
puts "Seed: #{seed}" unless seed.nil?
```

```
src : fish — Konsole
┌[X]-[07:24-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└-_6r4ndf4th3r$./task_7_random_array.rb
Generated 40 random integers in range 0..9:
  5   8   2   1   9
  8   8   2   2   6
  9   1   9   7   0
  9   1   0   7   2
  0   8   4   7   2
  6   7   4   7   3
  5   1   6   2   1
  3   4   3   0   8
┌[X]-[07:24-15/04]-[/home/_6r4ndf4th3r/TSI/PLC/LAB-4/src]
└-_6r4ndf4th3r$
```

3. CONCLUSIONS

In this laboratory work, seven console programs were implemented in Ruby to demonstrate string handling, array processing, matrix traversal, validation, formatting, and random data generation. Ruby made the solutions compact because strings and arrays are first-class objects with many built-in methods such as length, reverse, join, map, each_slice, and format.

Compared with C++, Ruby requires much less boilerplate for strings and dynamic arrays. In C++, the same tasks usually rely on std::string, std::vector, loops, and more explicit type management. Ruby is therefore faster to write for small educational tasks, while C++ gives stronger control over memory and performance.

Compared with Java, Ruby is more concise because Java uses fixed-size arrays, explicit class structure, and stronger type declarations. Compared with Python, Ruby is similarly productive, but Ruby's block-oriented syntax and methods such as each_slice make array-processing tasks especially readable. Overall, Ruby is well suited for this laboratory topic because its syntax makes string manipulation and array operations easy to express and compare with other programming languages.

4. REFERENCES

1. https://docs.ruby-lang.org/en/4.0/String.html
2. https://docs.ruby-lang.org/en/4.0/Array.html
3. https://docs.ruby-lang.org/en/4.0/Integer.html
4. https://docs.ruby-lang.org/en/4.0/Random.html
5. https://docs.ruby-lang.org/en/4.0/Kernel.html